

NOM – PRENOM :

On considère la liste suivante $L = [[1,4,5,3],[-6,3,2,1],[6,4,8,9]]$ que l'on peut représenter sous forme de matrice :

$$\begin{pmatrix} 1 & 4 & 5 & 3 \\ -6 & 3 & 2 & 1 \\ 6 & 4 & 8 & 9 \end{pmatrix}$$

1. Quelle commande permet d'obtenir le nombre de lignes de cette matrice ?
2. Quelle commande permet d'obtenir le nombre de colonnes de cette matrice ?
3. Que vaut L après chacune des commandes suivantes :

Commande	L
<code>L[2][3] = 3</code>	
<code>L.append([1,5,6,8])</code>	
<code>L[1].append(5)</code>	

Pour la suite, on suppose que la liste L est de nouveau celle du début de l'énoncé.

4. Quelle commande permet d'obtenir la valeur 6 contenue dans L ?
5. Écrire une fonction `affiche_valeurs(L)` qui affichera les différentes valeurs contenues dans L

```
>> 1 4 5 3 -6 3 2 1 6 4 8 9
```

```
def affiche_valeur(L) :
```

```
    |
```

6. Écrire une fonction `recherche_minimum(L)` qui renvoie sous forme de liste les indices `i` et `j` de la position du minimum ainsi que la valeur de ce minimum

Ici le minimum se trouve sur la deuxième ligne, première colonne et vaut -6

```
>> assert recherche_minimum(L) == [1,0,-6]
```

```
def recherche_minimum(L) :
```

```
|
```

7. On considère la liste suivante : `L = [[5,8,9],[2,4,3],[1,6,7]]`

Écrire une fonction `addition_colonne(L)` fait la somme des éléments des différentes colonnes

```
>> assert addition_colonne(L) == [8,18,19]
```

En effet pour la première colonne par exemple, on fait la somme : $5 + 2 + 1 = 8$

```
def addition_colonne(L) :
```

```
|
```

PROJETS LISTES

1. Écrire une fonction `make_liste(n)` qui renvoie une matrice carrée $n \times n$ contenant en ligne les valeurs de i^n pour i de 1 à n

$$\begin{pmatrix} 1^1 & \dots & 1^n \\ \vdots & \ddots & \vdots \\ n^1 & \dots & n^n \end{pmatrix}$$

```
>> assert make_liste(3) == [[1,1,1],[2,4,8],[3,9,27]]
```

```
def make_liste(n) :
```

2. Écrire une fonction `symetrie_matrice(L)` qui crée le « symétrique d'une matrice » carrée par rapport à un « axe » vertical qui passerait par la colonne du milieu si le nombre de colonnes est impair ou entre deux colonnes si le nombre de colonnes est pair

$$\begin{pmatrix} 2 & 3 & 4 \\ 8 & 4 & 7 \\ 3 & 1 & 0 \end{pmatrix} \Rightarrow \begin{pmatrix} 4 & 3 & 2 \\ 7 & 4 & 8 \\ 0 & 1 & 3 \end{pmatrix}$$

$$\begin{pmatrix} 7 & 9 \\ 3 & 4 \end{pmatrix} \Rightarrow \begin{pmatrix} 9 & 7 \\ 3 & 4 \end{pmatrix}$$

```
def symetrie_matrice(L) :
```

3. Simulation jeu de « bataille »

Vous connaissez certainement le jeu de la bataille. Les cartes d'un jeu de 52 cartes sont distribuées à deux joueurs de façon aléatoire. Simultanément chaque joueur montre la première carte de sa main et le joueur qui a la carte la plus forte l'emporte.

Les choses communes avec le jeu originel :

- Jeu de 52 cartes : 4 « couleurs » (Pique, Cœur, Carreau, Trèfle) de 13 « valeurs » (2 jusqu'à AS)
- Chaque joueur aura donc 26 cartes en main
- Chaque joueur révèle une carte au hasard.
- Cette carte est donc prélevée de son jeu et ne pourra donc plus être jouée
- Le gagnant est celui qui a la carte la plus forte

Nous apporterons quelques modifications :

- En cas d'égalité, c'est la « couleur » la plus forte qui l'emporte (Pique > Cœur > Carreau > Trèfle)
- Les joueurs marqueront un point chaque fois qu'il gagneront un affrontement
- Le joueur qui a le plus de point lorsque les 26 cartes sont jouées a gagné
- Les « valeurs » seront codées par des valeurs entre 2 et 14 (AS)
- Les « couleurs » seront codées par des valeurs entre 1 et 4
- Ainsi le Valet de Pique sera codé par [11,4]

Quelques contraintes :

- Une fonction `make_jeu()` qui crée une liste de 52 listes de deux éléments contenant une valeur et une couleur.

[[2,1], [3,1], [4,1], ... , [14,1],

[2,2], [3,2], [4,2], ... , [14,2],

[2,3], [3,3], [4,3], ... , [14,3],

[2,3], [3,3], [4,3], ... , [14,3]]

- Une fonction `check_winner(c1, c2)` qui renvoie le gagnant d'un affrontement entre deux cartes `c1` et `c2`.